

Loading API keys

```
In [ ]: import os
        from dotenv import load_dotenv

        # Load the variables from the .env file into os.environ
        load_dotenv()
```

Guardian

test the Guardian api key

```
In [7]: import requests
        import json
        from datetime import date

        # Build the query parameters
        query = "ukraine war"
        from_date = "2022-01-01" # Starting from 2022-01-01
        to_date = date.today().strftime("%Y-%m-%d") # Today's date in YYYY-MM-DD format

        # Construct the URL for the Guardian API search
        url = (
            "https://content.guardianapis.com/search"
            f"?q={query}"
            f"&from-date={from_date}"
            f"&to-date={to_date}"
            "&order-by=newest" # You can sort by newest or oldest
            f"&api-key={api_key}"
        )

        # Make the GET request
        response = requests.get(url)

        # Check the status code
        if response.status_code == 200:
            data = response.json()
            # Print out the total number of results
            total_results = data["response"]["total"]
            print(f"Total results: {total_results}")

            # Example: printing out the first few article titles and URLs
            results = data["response"]["results"]
            for article in results[:5]:
                print(f"- {article['webTitle']}")
                print(f"  URL: {article['webUrl']}")
                print()
```

```

else:
    print(f"Error: received status code {response.status_code}")

```

Total results: 27923

- Australia news live: David McBride appeal begins; housing downturn over after rate cut, data suggests

URL: <https://www.theguardian.com/australia-news/live/2025/mar/03/australia-news-live-politics-election-anthony-albanese-peter-dutton-cyclone-alfred-updates-oscars-ceremony-david-mcbride-appeal-act-supreme-court-ntwnfb>

- Zelenskyy says minerals deal is ready to be signed as he seeks to move conversation on from White House spat - live

URL: <https://www.theguardian.com/world/live/2025/mar/02/ukraine-war-volodymyr-zelensky-keir-starmer-donald-trump-us-europe-eu-russia-defence-latest-live-news>

- Keir Starmer says Europe 'at crossroads in history' and must support Ukraine

URL: <https://www.theguardian.com/world/2025/mar/02/keir-starmer-europe-crossroads-history-must-support-ukraine>

- Zelenskyy offers no apology to Trump and says row brought nothing positive

URL: <https://www.theguardian.com/world/2025/mar/02/zelenskyy-offers-no-apology-to-trump-as-he-says-row-brought-nothing-positive>

- Starmer struggles to remain upright under the weight of his contradictions | Zoe Williams

URL: <https://www.theguardian.com/politics/2025/mar/02/starmer-struggles-to-remain-upright-under-the-weight-of-his-contradictions>

Parse data from all relevant page

```

In [ ]: import requests
import json
from datetime import date, datetime
import os
import time

API_KEY = os.environ.get("Guardian_API_KEY")
query = "ukraine war"
from_date = "2022-01-01" # Start date
to_date = date.today().strftime("%Y-%m-%d") # End date (today)

# The maximum allowed page size by The Guardian API is 200
page_size = 200

# Dictionary to hold the results, with article's title as the key
# and a list [publication_date, url] as the value
all_results = {}

page = 1 # We start from page 1

while True:
    # Build the URL for the current page
    url = (
        "https://content.guardianapis.com/search"
        f"?q={query}"

```

```

        f"&from-date={from_date}"
        f"&to-date={to_date}"
        f"&order-by=newest"
        f"&page={page}"
        f"&page-size={page_size}" # <-- Note the hyphen here
        f"&api-key={API_KEY}"
    )

    response = requests.get(url)
    # wait for one second to avoid ratelimit
    time.sleep(1)
    if response.status_code != 200:
        print(f"Error: received status code {response.status_code}")
        break

    data = response.json()

    # Safety check in case the JSON doesn't have the expected structure
    if "response" not in data or "results" not in data["response"]:
        print("No valid 'response' or 'results' field in JSON.")
        break

    current_results = data["response"]["results"]

    # Add them to our dictionary
    for article in current_results:
        title = article["webTitle"]
        pub_date_str = article["webPublicationDate"] # e.g. "2023-02-10T12:34:56Z"

        # Convert to a datetime object
        # (ISO 8601 with trailing 'Z' means UTC, so we replace 'Z' with '+00:00')
        pub_date_dt = datetime.fromisoformat(pub_date_str.replace("Z", "+00:00"))

        # If you want to store just 'YYYY-MM-DD' as a string, do:
        pub_date_formatted = pub_date_dt.strftime("%Y-%m-%d")

        web_url = article["webUrl"]

        # Store in the dictionary: title -> [date, web_url]
        # NOTE: If multiple articles have the same title, they will overwrite each
        all_results[title] = [pub_date_formatted, web_url]

    current_page = data["response"].get("currentPage", 1)
    total_pages = data["response"].get("pages", 1)

    print(
        f"Fetches page {current_page}/{total_pages}. "
        f"Articles so far: {len(all_results)}",
        end="\n"
    )

    # Stop if we've reached the last page
    if current_page >= total_pages:
        break

    # Otherwise, go to the next page

```

```

    page += 1

# Once all pages are fetched, save results as a JSON file
    with open("../data/guardian_ukraine_war_headlines.json", "w", encoding="utf-8") as f:
        json.dump(all_results, f, ensure_ascii=False, indent=2)

    print(f"\nDone! Saved {len(all_results)} articles to guardian_ukraine_war_headlines

```

Fetches page 140/140. Articles so far: 27749

Done! Saved 27749 articles to guardian_ukraine_war_headlines.json

NYT

```

In [3]: import requests
import json
from datetime import date, datetime
import os
import time

NYT_API_KEY = os.environ.get("NYT_API_KEY")
query = "ukraine war"
from_date = "2022-01-01" # Start date
to_date = date.today().strftime("%Y-%m-%d") # End date (today)

# The maximum allowed page size by The Guardian API is 200
page_size = 200

# Dictionary to hold the results, with article's title as the key
# and a list [publication_date, url] as the value
all_results = {}

page = 1 # We start from page 1

url = (
    "https://api.nytimes.com/svc/search/v2/articlesearch.json"
    f"?q={query}"
    f"&api-key={NYT_API_KEY}"
    f"&begin_date={20220101}"
)

response = requests.get(url)

if response.status_code != 200:
    print(f"Error: received status code {response.status_code}")

data = response.json()

```

test NYT key

```

In [ ]: import requests
import json
from datetime import date
import time

NYT_API_KEY = os.environ.get("NYT_API_KEY")
query = "ukraine war"

```

```

# The New York Times Article Search API requires dates in YYYYMMDD format
begin_date = "20220101"
end_date = date.today().strftime("%Y%m%d")

all_articles = []
page = 0

while True:
    # Build the base URL and parameters for this page
    url = "https://api.nytimes.com/svc/search/v2/articlesearch.json"
    params = {
        "q": query,
        "begin_date": begin_date,
        "end_date": end_date,
        "sort": "newest",    # newest or oldest
        "page": page,        # 0-based page index
        "api-key": NYT_API_KEY,
    }

    response = requests.get(url, params=params)
    if response.status_code != 200:
        print(f"Error: received status code {response.status_code}")
        break

    data = response.json()

    # Extract the list of article docs
    docs = data.get("response", {}).get("docs", [])
    if not docs:
        # No more articles, so break
        break

    # Add to our cumulative list
    all_articles.extend(docs)

    # Print a quick status
    total_hits = data["response"]["meta"]["hits"] # total # of matching results
    print(f"Fetched page {page} | Articles so far: {len(all_articles)}", end="\r")

    # The maximum number of pages is hits/10, but the API also enforces a 200-page
    # If we've reached the last possible page or the 200-page limit, stop
    if page >= (total_hits // 10) or page >= 200:
        break

    page += 1

    time.sleep(12)

print(f"\n\nTotal articles fetched: {len(all_articles)}")

```

Fetched page 81 | Articles so far: 820

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[5], line 55
      51         break
      53     page += 1
--> 55     time.sleep(12)
      57 print(f"\n\nTotal articles fetched: {len(all_articles)}")
      59 # -----
      60 # Save to a JSON file
      61 # -----
      62
      63 # If you just want to store the raw 'docs', you can dump directly:

KeyboardInterrupt:

```

Parse NYT Monthly Archive then Search for keyword

```

In [ ]: import os
import json
import requests
from datetime import date, datetime

def fetch_and_save_monthly_archive(year, month, api_key, folder="nyt_archives"):
    """
    Fetch all articles for (year, month) from NYT Archive API and save as JSON.
    Returns the list of articles (docs).
    """
    # Ensure the output folder exists
    os.makedirs(folder, exist_ok=True)

    # API endpoint
    url = f"https://api.nytimes.com/svc/archive/v1/{year}/{month}.json"
    params = {"api-key": api_key}

    resp = requests.get(url, params=params)
    if resp.status_code != 200:
        print(f"Error fetching {year}-{month}: HTTP {resp.status_code}")
        return []

    data = resp.json()
    docs = data.get("response", {}).get("docs", [])

    # Build file path: e.g. "nyt_archives/nyt_archive_2022_01.json"
    filename = os.path.join(folder, f"nyt_archive_{year}_{month:02d}.json")

    with open(filename, "w", encoding="utf-8") as f:
        json.dump(docs, f, ensure_ascii=False, indent=2)

    print(f"Saved {len(docs)} articles to {filename}")
    return docs

def matches_keyword(doc, keyword="Russian Invasion of Ukraine (2022)":
    """
    Checks if an article doc contains the keyword in:

```

```

        - headline.main
        - abstract
        - lead_paragraph
        - or any of the doc['keywords']
Returns True if matched, else False.
"""

# Check headline
headline = doc.get("headline", {}).get("main", "") or ""
if keyword in headline:
    return True

# Check abstract
abstract = doc.get("abstract", "") or ""
if keyword in abstract:
    return True

# Check Lead paragraph
lead_para = doc.get("lead_paragraph", "") or ""
if keyword in lead_para:
    return True

# Check keywords (List of dictionaries)
kw_list = doc.get("keywords", [])
for kw in kw_list:
    # Typically kw['value'] is a string
    value = kw.get("value", "") or ""
    if keyword in value:
        return True

return False

def main():
    # 1. Fetch all articles from each month, Jan 2022 -> current month
    start_year = 2022
    start_month = 1

    today = date.today()
    end_year = today.year
    end_month = today.month

    for year in range(start_year, end_year + 1):
        # If this is the final year, only go up to current month
        last_month = 12 if (year < end_year) else end_month

        for month in range(start_month if year == start_year else 1, last_month + 1):
            # Fetch and save each month individually
            fetch_and_save_monthly_archive(year, month, NYT_API_KEY, folder="nyt_ar
            time.sleep(12)

    # 2. Now Load all the monthly files, search for "Russian Invasion of Ukraine (2
    # and collect articles into a single dict: {title: [YYYY-MM-DD, web_url], ..
    matched_articles = {}

    # We'll iterate again over the same year-month range, reading the saved JSON.
    for year in range(start_year, end_year + 1):
        last_month = 12 if (year < end_year) else end_month

```

```

for month in range(start_month if year == start_year else 1, last_month + 1):
    filename = os.path.join("nyt_archives", f"nyt_archive_{year}_{month:02d}")

    # If for some reason the file wasn't saved, skip
    if not os.path.exists(filename):
        continue

    with open(filename, "r", encoding="utf-8") as f:
        docs = json.load(f) # This is a list of article docs

    # Filter for relevant docs
    for doc in docs:
        if matches_keyword(doc, "Russian Invasion of Ukraine (2022)":
            # Build a unique title
            headline = doc.get("headline", {}).get("main", "Untitled")
            # Date in 'YYYY-MM-DD' format
            pub_date = doc.get("pub_date", "") # e.g. "2023-02-10T12:34:56"
            if pub_date:
                pub_date = pub_date[:10] # just grab 'YYYY-MM-DD'

            web_url = doc.get("web_url", "")

            # Store in dictionary: matched_articles[title] = [pub_date, web_url]
            # WARNING: If multiple articles share identical headlines, they
            matched_articles[headline] = [pub_date, web_url]

    # 3. Save all matched articles in a single JSON
    with open("../data/nyt_ukraine_war_headlines.json", "w", encoding="utf-8") as out_f:
        json.dump(matched_articles, out_f, ensure_ascii=False, indent=2)

    print(f"\nDone! Found {len(matched_articles)} articles containing 'Russian Invasion of Ukraine (2022)'")
    print("Saved results to 'nyt_ukraine_war_headlines.json'.")

if __name__ == "__main__":
    main()

```


Saved 3799 articles to nyt_archives\nyt_archive_2022_01.json
Saved 4059 articles to nyt_archives\nyt_archive_2022_02.json
Saved 4310 articles to nyt_archives\nyt_archive_2022_03.json
Saved 3934 articles to nyt_archives\nyt_archive_2022_04.json
Saved 4179 articles to nyt_archives\nyt_archive_2022_05.json
Saved 4374 articles to nyt_archives\nyt_archive_2022_06.json
Saved 3835 articles to nyt_archives\nyt_archive_2022_07.json
Saved 3983 articles to nyt_archives\nyt_archive_2022_08.json
Saved 4087 articles to nyt_archives\nyt_archive_2022_09.json
Saved 3857 articles to nyt_archives\nyt_archive_2022_10.json
Saved 4742 articles to nyt_archives\nyt_archive_2022_11.json
Saved 3548 articles to nyt_archives\nyt_archive_2022_12.json
Saved 3423 articles to nyt_archives\nyt_archive_2023_01.json
Saved 3260 articles to nyt_archives\nyt_archive_2023_02.json
Saved 5769 articles to nyt_archives\nyt_archive_2023_03.json
Saved 3718 articles to nyt_archives\nyt_archive_2023_04.json
Saved 3881 articles to nyt_archives\nyt_archive_2023_05.json
Saved 3788 articles to nyt_archives\nyt_archive_2023_06.json
Saved 3548 articles to nyt_archives\nyt_archive_2023_07.json
Saved 3668 articles to nyt_archives\nyt_archive_2023_08.json
Saved 3823 articles to nyt_archives\nyt_archive_2023_09.json
Saved 3906 articles to nyt_archives\nyt_archive_2023_10.json
Saved 3734 articles to nyt_archives\nyt_archive_2023_11.json
Saved 3525 articles to nyt_archives\nyt_archive_2023_12.json
Saved 3785 articles to nyt_archives\nyt_archive_2024_01.json
Saved 3791 articles to nyt_archives\nyt_archive_2024_02.json
Saved 4242 articles to nyt_archives\nyt_archive_2024_03.json
Saved 3954 articles to nyt_archives\nyt_archive_2024_04.json
Saved 4268 articles to nyt_archives\nyt_archive_2024_05.json
Saved 3847 articles to nyt_archives\nyt_archive_2024_06.json
Saved 4134 articles to nyt_archives\nyt_archive_2024_07.json
Saved 4047 articles to nyt_archives\nyt_archive_2024_08.json
Saved 4054 articles to nyt_archives\nyt_archive_2024_09.json
Saved 4306 articles to nyt_archives\nyt_archive_2024_10.json
Saved 4602 articles to nyt_archives\nyt_archive_2024_11.json
Saved 3666 articles to nyt_archives\nyt_archive_2024_12.json
Saved 4142 articles to nyt_archives\nyt_archive_2025_01.json
Saved 3828 articles to nyt_archives\nyt_archive_2025_02.json
Saved 477 articles to nyt_archives\nyt_archive_2025_03.json

Done! Found 7201 articles containing 'Russian Invasion of Ukraine (2022)'.
Saved results to 'nyt_russian_invasion_ukraine_2022_matched.json'.